

Understanding Move Semantics, RVO, and NRVO

1. Move Semantics in `std::vector` and Complex Types

- **Simple Types:** For types like `int`, the elements are directly stored in a contiguous block of memory, and any reallocation simply involves copying these values to a new memory block.
- **Complex Types:** For classes or structs that manage resources (e.g., dynamically allocated memory), `std::vector` will use move semantics during reallocation. The move constructor of each element is called, transferring ownership of resources (e.g., pointers to heap memory) to the new memory block, leaving the original objects in a valid but unspecified state.

2. RVO (Return Value Optimization)

- **RVO** is an optimization where the compiler avoids the creation of a temporary object when a function returns an object by value. Instead of creating a temporary object and then copying or moving it into the caller's storage, the compiler directly constructs the return value in the caller's storage.
- **C++17 and Mandatory RVO:** Since C++17, RVO is guaranteed in certain cases by the standard. Specifically, when a function returns a temporary object or an object created within the return statement itself, the compiler must elide the copy/move operation and construct the object directly in the caller's storage.

3. NRVO (Named Return Value Optimization)

- **NRVO** is similar to RVO but applies when the function returns a named local variable (not a temporary). The compiler can optimize the return by constructing the named local variable directly in the caller's storage, skipping the need for an intermediate temporary.
- **Not Guaranteed:** Unlike RVO since C++17, NRVO is not guaranteed by the standard. Whether or not NRVO is applied depends on the compiler and the specific circumstances in your code. Modern compilers are generally good at applying NRVO when possible, but you can't rely on it being applied in every situation.

Summary

- **Move Semantics:** Critical for optimizing operations in containers like `std::vector` when dealing with complex types.
- **RVO:** An optimization that eliminates unnecessary copying or moving when returning objects by value. Since C++17, RVO is guaranteed in certain cases.

- **NRVO**: Similar to RVO, but applies to named variables. It's a powerful optimization but not guaranteed by the C++ standard.